

CTF-Misc 题解：steganography

题目信息

- 题目名：8.steganography
- 分类：CTF-Misc
- 提示：steganography
- 题目描述：我把宝藏藏起来了，寻找宝藏！（提交 `dart{}` 内的内容即可）

一、题目初步分析

题目名称直接给出 `steganography`，说明这是一个**隐写题**。

一开始拿到的是一个名为 `steganography_challenge` 的文件，它**没有标准图片后缀**，因此首先考虑：

1. 文件伪装
2. 图片被拼接 / 篡改
3. 图片中继续藏有压缩包或文本
4. 多层隐写

二、文件头分析

使用 **010 Editor** 打开 `steganography_challenge`，观察十六进制内容。

1. 发现 PNG 文件头

在偏移 `0x23` 处发现标准 PNG 头：

```
1 | 89 50 4E 47 0D 0A 1A 0A
```

说明这个文件本质上是：

- 前面带了一段无关头部
- 后面嵌入了一张 PNG 图片

因此第一步操作是：

- 从 PNG 头开始提取
- 到 PNG 结束标记 `IEND` 为止
- 保存为独立 PNG 文件

三、提取 PNG 主体

1. PNG 结束标记

PNG 文件尾部标准结束块为：

```
1 | 49 45 4E 44 AE 42 60 82
```

在文件中定位到该标记后，将：

- 从 0x23 开始
- 到 IEND AE 42 60 82 结束

这一整段复制出来，保存为：

```
1 | main.png
```

2. 继续分析尾部附加数据

在 IEND 之后，文件后方还有少量附加数据。
在其中能看到 Unicode 形式的字符串：

```
1 | layer2.png
```

这说明题目存在**第二层隐藏内容**，但该部分本身并不是完整图片，而更像是提示信息或附加元数据。

四、修复 PNG 结构

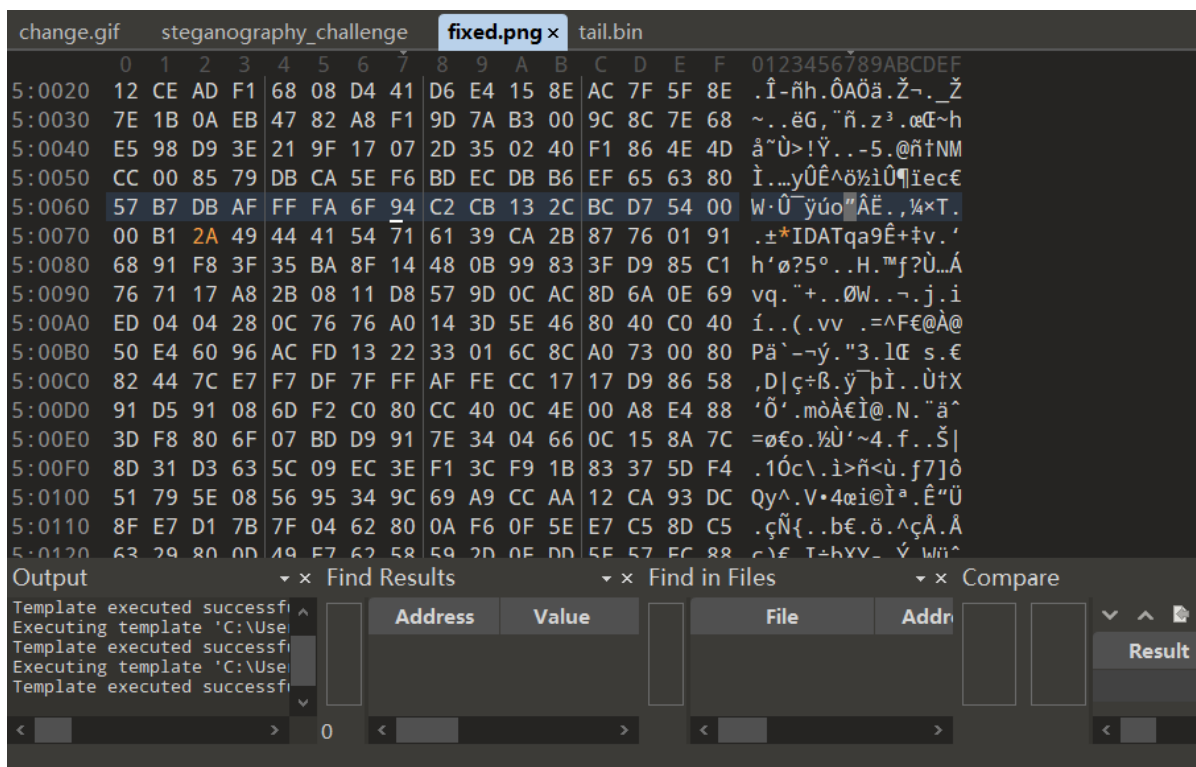
直接打开提取出的 PNG 时，图片无法被正常解码，说明其 PNG 结构存在人为破坏。
继续分析后发现是 IDAT 数据块长度字段异常。

需要在修复后的文件中手动修改以下 6 处长度字段。

修改位置与内容

偏移	修改后字节
0x21	00 01 00 03
0x10030	00 01 00 03
0x2003F	00 01 00 06
0x30051	00 01 00 03
0x40060	00 01 00 03
0x5006F	00 00 B1 2A

其中最后一处原本误判为 B127，后来改成 B12A 后，图片可以部分显示。



修复后的文件保存为：

1 fixed.png

五、提取图片最低位（LSB）数据

继续对 `fixed.png` 进行像素级分析。

最终确认：题目将一个 ZIP 文件藏在了图片的 **RGB 最低位（LSB）** 中。

提取思路

- 按像素顺序读取 `fixed.png`
- 取每个像素 RGB 三通道最低位
- 拼接成 bit 流
- 每 8 位还原成字节
- 前 4 字节表示压缩包长度（小端）
- 截取后续对应长度的数据，导出为 ZIP

提取脚本

```
1 from PIL import Image, ImageFile
2 import numpy as np
3
4 ImageFile.LOAD_TRUNCATED_IMAGES = True
5
6 img = Image.open("fixed.png")
7 img = img.convert("RGB")
8 arr = np.array(img)
9
10 bits = ((arr.reshape(-1, 3)) & 1).reshape(-1)
```

```

11
12 pad = (-len(bits)) % 8
13 if pad:
14     bits = np.concatenate([bits, np.zeros(pad, dtype=np.uint8)])
15
16 data = np.packbits(bits).tobytes()
17
18 size = int.from_bytes(data[:4], "little")
19 zip_data = data[4:4 + size]
20
21 with open("hidden.zip", "wb") as f:
22     f.write(zip_data)
23
24 print("done, size =", size)

```

成功导出：

```
1 | hidden.zip
```

七、分析 hidden.zip

解压 `hidden.zip` 后，得到如下文件：

```

1 | flag.zip
2 | pass1.zip
3 | pass2.zip
4 | pass3.zip
5 | pass4.zip
6 | pass5.zip
7 | pass6.zip

```

这说明题目并未直接给出 flag，而是通过多层压缩包和密码链继续隐藏。

名称	修改日期	类型	大小
 flag	2026/3/14 10:57	文件夹	
 flag	2026/1/5 7:16	压缩(zipped)文件...	1 KB
 pass1	2026/1/5 7:16	压缩(zipped)文件...	1 KB
 pass2	2026/1/5 7:16	压缩(zipped)文件...	1 KB
 pass3	2026/1/5 7:16	压缩(zipped)文件...	1 KB
 pass4	2026/1/5 7:16	压缩(zipped)文件...	1 KB
 pass5	2026/1/5 7:16	压缩(zipped)文件...	1 KB
 pass6	2026/1/5 7:16	压缩(zipped)文件...	1 KB

八、提取 6 个密码片段

分别打开 `pass1.zip ~ pass6.zip`，每个压缩包内都有一个被加密的文本文件。
进一步分析后，6 个压缩包分别给出 6 段密码片段：

```
1 pass
2 is
3 c1!x
4 xtLf
5 %fXY
6 PkaA
```

将其按顺序拼接，得到 `flag.zip` 的密码：

```
1 c1!xtLf%fXYPkaA
```

九、解压 flag.zip

使用上一步得到的密码解压 `flag.zip`，得到：

```
1 flag.txt
```

但 `flag.txt` 中并不是直接可见的 flag，只看到类似：

```
1 flag is here
```

而真正的 flag 仍然没有直接显示。

十、分析 flag.txt 中的零宽字符

继续检查 `flag.txt`，发现其中在可见文本后面跟着一串**不可见字符**。
这些字符并不是空格，而是典型的**零宽字符隐写**。

主要涉及两种字符：

- `U+200B`：零宽空格
- `U+200C`：零宽非连接符

映射规则如下：

- `U+200B` → 0
- `U+200C` → 1

解码过程

1. 提取可见文本后方所有零宽字符
2. 根据映射转为二进制串
3. 每 8 位分组
4. 转成 ASCII 文本

最终恢复出 flag：

```
1 dart{bf4100d9-cc8d-48f6-a095-54cbfad189e1}
```